

Dijkstra's Algorithm in Java

Mostafa Faik

October 27, 2024

Introduction

This assignment focuses on improving our previous solution for finding the shortest paths in a railroad network by implementing Dijkstra's algorithm. Previously, we implemented a depth-first search (DFS) approach, which was effective in smaller contexts but struggled with efficiency in larger graphs. Dijkstra's algorithm optimizes this by remembering where it has been and avoids re-exploring paths unnecessarily. This not only provides a faster solution for individual paths but also calculates the shortest path from a single source to all other nodes in the network. Through this assignment, we explore graph structures, benchmark the algorithm's efficiency, and compare its performance with the DFS approach.

Defining the Graph Structure

Our graph structure represents cities and the train connections between them. As in the previous assignment, each City holds a name, a unique identifier, and a list of its direct connections (neighbors). Each Connection has two parts: the destination city and the time (distance) in minutes it takes to reach this city from the current city, which simplifies the task of adding each neighboring city to our search, since we only need to store these two properties. This setup creates a robust and flexible graph structure to support Dijkstra's algorithm.

Each city is assigned a unique identifier that allows quick access and organization in arrays. With Dijkstra's algorithm, this identifier is especially useful as it helps to track the shortest known paths (stored in the done array) efficiently and making lookups faster and memory-efficient when storing shortest paths.

Implementing a Path and Tracking Progress

A `Path` object tracks the current city, the previous city, and the cumulative distance from the origin. This setup ensures that we always have the information needed to trace back the shortest path. In addition to the `Path` structure, a `done` array is used to store the shortest path to each city once found. The `done` array's index corresponds to the city's identifier, simplifying access and ensuring each city's shortest path is stored only once.

The `Done` array works by recording the shortest path to each city, identified by its ID. Once we confirm the shortest path to a city, we add it to the `Done` array. Any subsequent attempts to add a longer path to the same city are ignored, helping us avoid redundant processing.

Using a Priority Queue

Dijkstra's algorithm requires expanding paths in order of their cumulative distances, selecting the shortest path each time. This is managed by a priority queue that orders paths by distance, allowing the algorithm to always explore the closest path first. By implementing this queue, we avoid unnecessary processing of longer paths until absolutely needed, minimizing search time and improving efficiency.

Dijkstra's Algorithm Implementation

The core of Dijkstra's algorithm is its iterative approach to expanding paths in order of shortest cumulative distance. Starting from the source city, it proceeds as follows:

- Initialization: Begin with the source city, treating it as having a distance of zero. Add this path to the priority queue.
- Processing Paths: Repeatedly take the path with the shortest distance from the priority queue:
 - If the destination city is reached, store this as the shortest path in the `Done` array.
 - If this is the first time reaching a city with the shortest distance, add it to the `Done` array and expand all paths to its neighboring cities.
- Expanding Neighboring Paths: For each neighbor, create a new path entry with an updated cumulative distance and add it to the priority queue (unless the city is already recorded in `Done`).

- **Completion:** The algorithm stops when we reach the destination or when there are no more paths to explore in the queue.

By recording each **shortest path** in **Done**, we avoid re-processing cities, which cuts down on redundant **path** expansions and speeds up the **search**.

Comparison with DFS Approach

The previous DFS-based method faced issues with repetitive **path searches**. It often revisited cities unnecessarily, leading to increased **runtime** for long distances or complex networks. Dijkstra's algorithm, however, efficiently avoids these redundancies by marking each city once its **shortest path** is confirmed. This leads to significant performance improvements, especially over large networks like those in Europe.

Route	DFS (ms)	Dijkstra's (ms)	Shortest Path (min)
Malmö to Kiruna	807	421	1162
Malmö to Stockholm	424	208	273
Malmö to Göteborg	313	78	153

Table 1: DFS Approach vs Dijkstra Algorithm

We can observe that, Dijkstra's algorithm consistently outperformed DFS, especially for longer paths. For example, the path from Malmö to Stockholm took *208 ms* with Dijkstra's, compared to *424 ms* with DFS.

These numbers demonstrate the improved efficiency of Dijkstra's algorithm, especially on long-distance or complex routes. Dijkstra's approach quickly identifies shortest paths while avoiding unnecessary searches.

Benchmarking Shortest Paths from Malmö

We also benchmarked the time taken by Dijkstra's algorithm to compute the **shortest path** from Malmö to twelve other cities in Europe. The table includes the **path distance**, the number of cities involved in the search (as recorded in **Done**), and the time taken:

To	Shortest Path (min)	Processed Cities	Time Taken (ms)
Stockholm	273	35	208
Göteborg	153	20	78
Kiruna	1162	116	806
Oslo	381	50	384
Köpenhamn	43	5	17
Paris	775	82	150
Berlin	424	57	79
Rom	1337	123	109
Amsterdam	623	69	63
Wien	924	92	241
Madrid	1347	125	145
London	795	84	97

Table 2: Benchmarking shortest paths from Malmö to various cities using Dijkstra’s algorithm.

Time Complexity Analysis

DFS Complexity

The DFS approach lacks optimal path selection, resulting in a higher complexity as it explores all possible paths. DFS does not account for edge weights, and each node may be revisited multiple times, resulting in exponential complexity for weighted graphs. This limits its practicality for finding shortest paths, particularly in large graphs.

Dijkstra’s Algorithm Complexity

Dijkstra’s algorithm operates with a time complexity of $O((V+E)\log V)$:

- V represents the number of vertices (cities), while E represents the number of edges (connections).
- The priority queue (used to select the shortest path) contributes the $\log V$ factor for each operation.

The algorithm’s efficiency arises from the priority queue, which keeps path expansions ordered by cumulative distance, minimizing unnecessary paths. Each city is processed only once, reducing overall work compared to DFS, which often re-evaluated paths multiple times. This performance difference becomes especially clear as the number of cities or connections increases.

Conclusion

In this assignment, Dijkstra's algorithm has proven to be far more efficient for finding shortest paths in large graphs than the DFS-based approach. Dijkstra's intelligent use of memory allowed us to avoid redundant path expansions, leading to much faster performance across various routes. The benchmark results demonstrate substantial runtime reductions, making Dijkstra's algorithm the clear choice for complex networks and larger datasets. By comparing these two approaches, we conclude that Dijkstra's algorithm offers a robust solution for shortest-path problems, providing both scalability and efficiency essential for larger graphs. The assignment has also highlighted the importance of using optimized algorithms and data structures when handling real-world network problems like route planning.